Rust - Vecteurs

Vecteurs en Rust (`Vec<T>`)

📌 Table des Matières

1. Introduction aux vecteurs
2. Création d'un vecteur
3. Mise à jour d'un vecteur
4. Lecture des éléments
5. Itération sur un vecteur
6. Gestion de la capacité
7. Vecteurs et énumérations
8. Libération de la mémoire
9. Méthodes complémentaires parmi les plus couramment utilisées
10. Exemple complet : Gestionnaire bibliographique
11. Ressources et bonnes pratiques

Introduction aux vecteurs

Les vecteurs (`Vec<T>`) sont des types de tableaux dynamiques qui :

- Stockent plusieurs valeurs du même type, de façon contiguë, en mémoire.
- Sont alloués sur le tas (*heap*) : permettant de gérer des tailles variables et indéfinies.
- Redimensionnables : peuvent grandir ou rétrécir dynamiquement.
- `Vec<T>` = notation générique où `T` est le type des éléments stockés (ex: `Vec<i32>` pour un vecteur d'entiers).
- Sont idéaux pour les listes d'éléments (ex: liste de références bibliographiques)

```rust
// Exemple: Liste d'ISBN dans un gestionnaire bibliographique
let isbn_list: Vec<String> = Vec::new();
```

Les vecteurs ont un accès par index en temps constant (*O(1)*), ainsi que pour un ajout à la fin et un retrait depuis la fin, en raison de l'accès de façon contiguë en mémoire.

Toutefois l'ajout (mais cas rare) peut s'avérer coûteux (*O(n)*), si le vecteur est plein et qu'il est alors nécessaire de réallouer un espace mémoire plus grand.

```
fn main() {
    let mut vector = vec![100, 32, 57, 15, 102, 1972];
    let item_from_vector = vector[2]; // Accès par index
    let slice_from_vector = &vector[2..4]; // type slice &[i32]
    // slice = vue sur une tranche du vecteur par référence
    println!("{}", item_from_vector); // 57
    println!("{:?}", slice_from_vector); // [57, 15]
    // {:?} formatage en mode Debug (pour afficher des structures de
données)
    for i in slice_from_vector {
        println!("{i}"); // Chaque élément de slice_from_vector
    }
    vector.push(30); // Ajoute élément en fin
    // slice_from_vector n'est plus accessible,
    // car référence une partie du vecteur invalide en mémoire
    println!("{:?}", vector); // [100, 32, 57, 15, 102, 1972, 30]
    println!("{:?}", vector.pop()); // Suppression du dernier élément -
Some(30)
    println!("{:?}", vector); // [100, 32, 57, 15, 102, 1972]
}
```

Les vecteurs garantissent qu'ils n'allouent jamais plus que `usize::MAX` octets. C'est la valeur maximale que peut prendre un entier signé de la taille d'un pointeur sur votre système : évite les débordements de mémoire, garantit que le code fonctionne sur toutes les plateformes (y compris avec une mémoire physique limitée). En pratique, cette limite est extrêmement grande (des milliards de gigaoctets) et ne sera jamais atteinte dans des applications classiques.

Création d'un vecteur

1. Vecteur vide

```
let mon_vecteur: Vec<String> = Vec::new();
// ou :
let mon_vecteur = Vec::<String>::new(); // syntaxe dite 'turbofish'
```

L'annotation de type est obligatoire si le compilateur ne peut déterminer le type (ne peut inférer le type) des éléments contenus.

Sans annotation de type :

```
fn main() {
    let mut vec = Vec::new(); // Type non précisé.
    vec.push(42); // Le compilateur déduit que `vec` est un `Vec<i32>`.
    println!("{:?}", vec); // Affiche [42].
}
```

Ici, `Vec::new()` crée un vecteur vide sans type explicite, mais le compilateur déduit que c'est un `Vec<i32>` grâce à `vec.push(42)`. A noter également que qu'avec `Vec::new()`, Le vecteur n'allouera pas de mémoire (cf. infra) tant que des éléments ne lui sont pas ajoutés avec la méthode `.push()`.

2. Vecteur avec valeurs initiales (type inféré)

```
let auteurs = vec![
    String::from("Hemingway"),
    String::from("Kerouac"),
    String::from("Nabokov")
];
```

3. Avec la macro `vec!`

```
let vector: <i32> = vec![] // Vecteur vide - Annotation de type
obligatoire
let annees_publication = vec![1941, 1980, 1979]; // A partir d'une
liste d'éléments
```

4. Avec une chaîne d'itérateurs (*Iterator chain*)

Imaginons prendre chaque élément d'un tableau, les transformer et les insérés dans un vecteur :

```
fn main() {
    let array = ["hemingway", "djian", "kerouac"];
    let authors: Vec<_> = array.iter().map(|x|
x.to_uppercase()).collect();
```

```
    println!("{:?}", authors); // ["HEMINGWAY", "DJIAN", "KEROUAC"]
}
```

On peut chaîner plusieurs méthodes sur un itérateur pour former une *pipeline* de transformations. A noter que le *pattern* `map().collect()` est très courant en Rust (pratique standard et recommandée).

Mise à jour d'un vecteur

Ajouter des éléments avec `.push()`

Ajoute un élément à la fin du vecteur.

```
let mut bibliographie = Vec::new();
bibliographie.push(String::from("Pour Qui Sonne Le Glas"));
bibliographie.push(String::from("Machenka"));
```

En moyenne, `.push()` est constant en temps (*O(1)*).

Attention : les vecteurs doivent être mutables

```
let mut titres = vec![String::from("Stephen le Héros")];
titres.push(String::from("Entre Ciel et Terre"));  // OK

let titres_immuable = vec![String::from("1984")];
titres_immuable.push(String::from("Fahrenheit 451"));  // ❌ Erreur de
compilation
```

Insertion

La méthode `.insert(index, element)` permet d'insérer `element` à la position `index` dans le vecteur, en décalant tous les éléments suivants vers la droite.

```
fn main() {
    let mut alphabet = vec!['A', 'C', 'D'];
    alphabet.insert(1, 'B');
    println!("{:?}", alphabet); // ['A', 'B', 'C', 'D']
}
```

Attention : *Panic* si `index > len`.

Lecture des éléments

1. Accès par indice

Les vecteurs implémentent le trait `Index`, ce qui permet d'accéder aux éléments par leur position (*index*) à l'aide de la syntaxe `[]` :

```
let authors = vec!["hemingway", "djian", "kerouac"];
println!("{}", authors[1]); // "djian"]
```

Panic en cas d'*index* invalide (hors limite).

2. Méthodes `.get()` et `.get_mut()`

Pour éviter les *panic*, utiliser `.get()` (accès immuable) ou `.get_mut()` (accès mutable) :

```
fn main() {
    let mut numbers = vec![10, 20, 30, 40, 50];

    // .get() : retourne Option<&T> (Some(&valeur) ou None) - Gestion
sécurisée
    match numbers.get(10) {
        Some(&value) => println!("Valeur à {}: {}", index, value),
        None => println!("L'index {} est hors limites !", index), //
cas ici
    }

    // Modification sécurisée d'un item (Retourne une référence
mutable)
    // ! Le vecteur doit être mutable
    if let Some(value) = numbers.get_mut(1) {
        *value = 200;
        println!("{:?}", numbers); // [10, 200, 30, 40, 50]
    }
}
```

3. Règles d'emprunt

Une *slice* (`&[T]`) est une vue sur une partie contiguë d'un tableau ou d'un `Vec`. Les slices sont immuables (*read-only*) et ne possèdent pas les données (ou propriété):

elles empruntent (*borrow) les données d'un `Vec` ou d'un tableau.

```
let mut livres = vec![String::from("L'Étranger")];
let premier = &livres[0];  // Emprunt immuable
livres.push(String::from("La Peste"));  // ❌ Erreur: emprunt mutable
alors qu'il y a un emprunt immuable
```

Ici, `&livres[0]` crée une slice qui ne couvre que le premier élément du vecteur, `&livres` aurait couvert tout le vecteur.

A noter qu'en Rust, on préfère passer des slices plutôt que des `Vec` en argument de fonction pour éviter les copies inutiles.

Itération sur un vecteur

Les itérateurs (`iter()` , `iter_mut()` , `into_iter()`) parcourent toujours les éléments dans l'ordre.

1. Itération immuable

```
for livre in &bibliographie {
    println!("Livre: {}", livre);
}
```

2. Itération mutable

```
let mut annees = vec![1941, 1980, 1979];
for annee in &mut annees {
    *annee += 1;  // Incrémente chaque année
}
```

3. Itération avec indices

```
for (index, livre) in bibliographie.iter().enumerate() {
    println!("Livre {}: {}", index + 1, livre);
}
```

Gestion de la capacité

Capacité *vs* longueur dans les vecteurs :

Terme	Définition	Code
Longueur	Nombre d'éléments actuellement stockés dans le vecteur.	`vec![1,2,3].len() // 3`
Capacité	Espace mémoire alloué pour les éléments futurs.	`Vec::with_capacity(10)`

```
let mut v = Vec::with_capacity(10); // Capacité = 10, Longueur = 0
v.push(1);                          // Capacité = 10, Longueur = 1
```

1. Pré-alouer de la mémoire avec `with_capacity`

Créer un nouveau vecteur vide avec une capacité initiale spécifiée.

```
let mut grande_bibliotheque = Vec::with_capacity(1000);  // Alloue pour
1000 éléments (capacité minimale garantie)
```

Conseillé si l'on connaît la taille approximative du vecteur :

- Évite les réallocations coûteuses (cf. infra).
- Meilleures performances pour les grands vecteurs
- Mémoire contiguë garantie

2. Mécanisme de réallocation

Quand la longueur dépasse la capacité, Rust doit :

1. Allouer un nouvel espace mémoire (généralement deux plus grand)
2. Copier tous les éléments existants vers le nouvel espace (opération coûteuse : `O(n)`)
3. Libérer l'ancien espace

```
let mut v = Vec::with_capacity(2);  // Capacité initiale = 2
v.push(1);  // Longueur = 1
v.push(2);  // Longueur = 2
v.push(3);  // Réallocation ! Capacité passe à 4 (×2)
```

`capacity() >= len()` est toujours vrai. Un vecteur n'allouera jamais moins que ce qu'il contient. Cela implique qu'il n'y a pas de réallocation inutile (si `len() <=`

`capacity()`). Par contre il est possible de rencontre une *Panic*, si la nouvelle capacité est supérieur à `isize::MAX` *bytes*, soit, environ 2^63 octets sur les systèmes 64-bit (valeur toutefois très importante - cas extrêmement rare).

Quand un vecteur réalloue (grandit), toutes les références existantes deviennent invalides. Cela est dû au fait que les éléments sont déplacés vers un nouvel emplacement mémoire. Il est donc préférable de ne pas stocker des références (à long terme) vers des éléments d'un vecteur. Il sera plus judicieux d'utilisez des indices plutôt que des références si le vecteur est susceptible de grandir.

Il est toutefois possible de garantir une stabilité des pointeurs entre réallocations. Tant qu'il n'y a pas de réallocation, les références vers les éléments restent valides, car des opérations comme `.push()` ou `.pop()` ne réallouent pas si `len() < capacity()`.

```
let mut v = Vec::with_capacity(10);
v.push(1);
v.push(2);
let r = &v[0];  // Référence valide
v.push(3);      // Pas de réallocation (capacité = 10 > len() = 3)
println!("{}", r);  // Sûr : car pas de réallocation
```

3. Réserver de la capacité

Un vecteur peut réserver plus d'espace que demandé pour éviter les réallocations fréquentes. Après l'appel à la méthode `.reserve(additionnel)`, la capacité sera supérieure ou égale à `self.len() + additionnel`. Par contre, si la capacité est suffisante, cela ne fait rien.

```
fn main() {
    let mut livres = vec!["Paris est une fête"];
    println!("{}", livres.capacity()); // 1
    livres.reserve(10);
    println!("{}", livres.capacity()); // 11
    livres.reserve(10);
    println!("{}", livres.capacity()); // 11, car capacité déjà
suffisante
}
```

4. Réduire la capacité

La méthode `.shrink_to_fit` réduit la capacité du vecteur au maximum possible.

```
let mut livres = vec![String::from("Livre 1"); 1000];
livres.shrink_to_fit();  // Libère l'espace inutilisé
```

Attention cependant : le comportement de cette méthode dépend de l'allocateur, qui peut soit :

- réduire la capacité du vecteur sur place (sans réallocation),
- réallouer le vecteur pour ajuster sa taille.

Savoir si une réallocation a eu lieu :

```
println!("Avant : capacité = {}, longueur = {}", v.capacity(),
v.len());
v.shrink_to_fit();
println!("Après : capacité = {}, longueur = {}", v.capacity(),
v.len());
```

A noter que généralement, avec l'allocateur par défaut, la méthode `.shrink_to_fit()` réduit la capacité sur place, sans réallocation.

Il est conseillé d'utiliser la méthode `.shrink_to_fit()` quand nous sommes sûr que le vecteur n'aura plus besoin de grandir, pour libérer de la mémoire.

Analogie : avec `shrink_to_fit()` c'est comme demander à un restaurant de réduire la taille de la table après le départ de certains des convives. Soit nous pouvons simplement enlever des chaises (réduction sur place), soit nous devrons changer de table pour une table plus petite (réallocation).

Il existe également la méthode `.shrink_to()` qui réduit la capacité d'un vecteur, mais garantit une capacité minimale spécifiée.

```
let mut v = Vec::with_capacity(1000);
v.shrink_to(100); // Réduit la capacité, mais garde au moins 100
emplacements
```

Cela va permettre de libérer de la mémoire tout en gardant une marge.

Dans le cas où nous demanderions la réduction de la taille de la capacité à une taille inférieure au nombre d'éléments que contient le vecteur :

```rust
let mut v = Vec::with_capacity(10);  // Capacité = 10, Longueur = 0
v.extend([1, 2, 3, 4, 5]);           // Longueur = 5, Capacité = 10
v.shrink_to(3);
```

La capacité finale sera déterminée par `nouvelle_capacité = max(capacité_spécifiée, longueur_actuelle)` soit entre `3` et `5`, et donc elle sera de `5`.

Vecteurs et énumérations

Pour stocker différents types dans un vecteur, utiliser une énumération:

```rust
#[derive(Debug)]
enum ReferenceBibliographique {
    Livre { titre: String, auteur: String, annee: u32 },
    Article { titre: String, revue: String, volume: u32 },
    These { titre: String, universite: String, annee: u32 },
}

let bibliographie = vec![
    ReferenceBibliographique::Livre {
        titre: String::from("Pêcheur d'Islande"),
        auteur: String::from("Pierre Loti"),
        annee: 1886,
    },
    ReferenceBibliographique::Article {
        titre: String::from("Les Carnets de Casteret"),
        revue: String::from("Pyrénées"),
        volume: 235,
    },
];
```

Utilisation

```rust
for ref_bib in &bibliographie {
    println!("{}", ref_bib);
}
```

Libération de la mémoire

Quand un vecteur est libéré (sort de la portée), ses éléments sont détruits dans l'ordre (du premier au dernier).

```
{
    let catalogues = vec![
        String::from("Philosophie"),
        String::from("Littérature"),
    ];
    // Utilisation des catalogues...
} // <- Les catalogues et leurs String sont libérés ici
```

Méthodes complémentaires parmi les plus couramment utilisées

Méthode	Fonction	Exemple d'utilisation
Création et capacité		
`try_with_capacity`	Crée un vecteur vide avec une capacité initiale, retourne `Result` en cas d'erreur d'allocation.	`let mut v = Vec::try_with_capacity(10).unwrap();`
`reserve_exact`	Réserve exactement la capacité supplémentaire demandée (sans sur-allocation).	`v.reserve_exact(10);`
`try_reserve`	Réserve une capacité supplémentaire, retourne `Result` en cas d'erreur.	`v.try_reserve(10).unwrap();`
`try_reserve_exact`	Réserve exactement la capacité	`v.try_reserve_exact(10).unwrap();`

Méthode	Fonction	Exemple d'utilisation
	demandée, retourne `Result` en cas d'erreur.	
`spare_capacity_mut`	Retourne un espace mutable pour ajouter des éléments sans réallocation.	`let space = v.spare_capacity_mut();`
Ajout et suppression		
`remove`	Retire et retourne l'élément à un index donné, en décalant les éléments suivants.	`let elem = v.remove(2);`
`swap_remove`	Retire un élément à un index donné en le remplaçant par le dernier élément.	`let elem = v.swap_remove(2);`
`truncate`	Réduit la longueur du vecteur à une nouvelle taille.	`v.truncate(5);`
`clear`	Vide le vecteur (longueur = 0, mais conserve la capacité).	`v.clear();`
`append`	Ajoute tous les éléments d'un autre vecteur à la fin.	`v1.append(&mut v2);`
`drain`	Retire une plage d'éléments et	`let drained: Vec<_> = v.drain(2..5).collect();`

Méthode	Fonction	Exemple d'utilisation
	retourne un itérateur.	
`retain`	Conserve uniquement les éléments satisfaisant une condition.	`numbers.retain(|&x| x % 2 == 0);` `Conserver uniquement les nombres pa`
`retain_mut`	Conserve les éléments satisfaisant une condition, avec accès mutable.	
`dedup`	Supprime les éléments consécutifs égaux.	`v.dedup();`
`dedup_by`	Supprime les éléments consécutifs égaux selon une fonction.	`nombres.dedup_by(|a, b| a == b);` `Supprimer les doublons consécutifs`
`dedup_by_key`	Supprime les éléments consécutifs égaux selon une clé.	`utilisateurs.dedup_by_key(|utilisat` `utilisateur.email.clone()); // Suppr` `les doublons consécutifs basés sur`
Accès et conversion		
`as_slice`	Retourne une vue immuable du vecteur sous forme de slice.	`let slice = v.as_slice();`
`as_mut_slice`	Retourne une vue mutable du vecteur sous forme de slice.	`let mut_slice = v.as_mut_slice();`
`as_ptr`	Retourne un pointeur vers le	`let ptr = v.as_ptr();`

Méthode	Fonction	Exemple d'utilisation
	premier élément.	
`as_mut_ptr`	Retourne un pointeur mutable vers le premier élément.	`let mut_ptr = v.as_mut_ptr();`
`first`	Retourne une référence au premier élément (retourne `Option`).	`let first = v.first();`
`first_mut`	Retourne une référence mutable au premier élément (retourne `Option`).	`let first_mut = v.first_mut();`
`last`	Retourne une référence au dernier élément (retourne `Option`).	`let last = v.last();`
`last_mut`	Retourne une référence mutable au dernier élément (retourne `Option`).	`let last_mut = v.last_mut();`
`into_boxed_slice`	Convertit le vecteur en une `Box<[T]>` (slice boxée).	`let boxed_slice = v.into_boxed_slice`
Tri et recherche		

Méthode	Fonction	Exemple d'utilisation
`sort`	Trie les éléments du vecteur.	`v.sort();`
`sort_by`	Trie les éléments selon une fonction de comparaison.	`nombres.sort_by(\\|a, b\\| a.cmp(b));` `Tri croissant (équivalent à sort(), explicite)`
`sort_by_key`	Trie les éléments selon une clé extraite.	`mots.sort_by_key(\\|mot\\| mot.len()); Tri par longueur des mots`
`binary_search`	Effectue une recherche binaire sur un vecteur trié.	`let index = v.binary_search(&42);`
`binary_search_by`	Recherche binaire avec une fonction de comparaison.	`match nombres.binary_search_by(\\|&x\ x.cmp(&7)) { ... } // Recherche de l valeur 7`
`binary_search_by_key`	Recherche binaire selon une clé extraite.	`match elements.binary_search_by_key( \\|&(id, _)\\| id) { ... } // Recherch l'élément avec id = 5`
Manipulations avancées		
`split_off`	Détache une partie du vecteur à partir d'un index et la retourne comme un nouveau vecteur.	`let split_vec = v.split_off(3);`
`resize`	Redimensionne le vecteur en ajoutant ou supprimant des éléments.	`v.resize(10, 0);`
`resize_with`	Redimensionne le vecteur en	`vec.resize_with(5, \\|\\| 0); // Ajout zéros`

Méthode	Fonction	Exemple d'utilisation
	utilisant une closure pour créer de nouveaux éléments.	
`extend_from_slice`	Ajoute les éléments d'une slice à la fin du vecteur.	`v.extend_from_slice(&[1, 2, 3]);`
`extend`	Ajoute les éléments d'un itérateur à la fin du vecteur.	`v.extend([1, 2, 3].iter());`
`concat`	Concatène les éléments d'un itérateur de vecteurs.	`let concatenated = vec![vec![1, 2], [3, 4]].concat();`
`join`	Concatène les éléments en utilisant un séparateur.	`let joined = v.join(", ");`

Exemple d'utilisation

```
/// Structure représentant un livre de la bibliothèque
#[derive(Debug)] // Permet d'utiliser {:?} pour l'affichage de débogage
struct Livre {
    titre: String,          // Titre du livre (String car la taille
peut varier)
    auteur: String,         // Idem avec le nom de l'auteur
    annee: u32,             // Année de publication (u32 car toujours
positif)
    mots_cles: Vec<String>, // Liste de mots-clés (Vec car nombre
variable)
}

impl Livre {
    /// Crée une nouvelle instance de Livre avec des valeurs par défaut
    fn nouveau() -> Self {
```

```
        Livre {
            titre: String::new(), // Chaîne vide
            auteur: String::new(),
            annee: 0,              // Valeur par défaut
            mots_cles: Vec::new(), // Vecteur vide
        }
    }

    /// Définit le titre du livre
    fn definir_titre(&mut self, titre: String) {
        self.titre = titre; // Simple remplacement de la valeur par
défaut
    }

    /// Définit l'auteur du livre
    fn ajouter_auteur(&mut self, auteur: String) {
        self.auteur = auteur;
    }

    /// Ajoute un mot-clé à la liste des mots-clés
    fn ajouter_mot_cle(&mut self, mot: String) {
        self.mots_cles.push(mot); // push() ajoute un élément à la fin
du vecteur
    }

    /// Définit l'année de publication
    fn definir_annee(&mut self, annee: u32) {
        self.annee = annee;
    }
}

fn main() {
    // Création d'un vecteur pour stocker nos livres
    // Vec<Livre> signifie "un vecteur contenant des éléments de type
Livre"
    let mut bibliotheque: Vec<Livre> = Vec::new();

    // --- Ajout du premier livre ---
    let mut livre1 = Livre::nouveau(); // Création d'une nouvelle
instance
    livre1.definir_titre(String::from("Pour Qui Sonne le Glas"));
    livre1.ajouter_auteur(String::from("Ernest Hemingway"));
```

```
    livre1.definir_annee(1940);
    livre1.ajouter_mot_cle(String::from("guerre"));
    livre1.ajouter_mot_cle(String::from("roman"));

    // Ajout du livre au vecteur bibliotheque
    // push() ajoute l'élément à la fin du vecteur
    bibliotheque.push(livre1);

    // --- Ajout du second livre ---
    let mut livre2 = Livre::nouveau();
    livre2.definir_titre(String::from("Le Baron perché"));
    livre2.ajouter_auteur(String::from("Italo Calvino"));
    livre2.definir_annee(1957);
    livre2.ajouter_mot_cle(String::from("fantaisie"));
    livre2.ajouter_mot_cle(String::from("philosophique"));

    bibliotheque.push(livre2); // Ajout au vecteur

    // --- Affichage de tous les livres ---
    println!("\nTous les livres de la bibliothèque:\n");
    // iter() crée un itérateur sur le vecteur
    // enumerate() ajoute un compteur (index) à chaque élément
    for (index, livre) in bibliotheque.iter().enumerate() {
        println!(
            "Livre {}:\n'{}' ({}) par {}\nMots-clés: {:?}\n",
            index + 1, // index commence à 0, donc on ajoute 1
            livre.titre,
            livre.annee,
            livre.auteur,
            livre.mots_cles // {:?} utilise Debug pour afficher le
vecteur
        );
    }
    // Note sur la gestion mémoire:
    // 1. Le vecteur 'bibliotheque' est alloué sur le tas (heap)
    // 2. Chaque Livre est stocké contiguëment en mémoire
    // 3. Quand 'bibliotheque' sortira de portée (à la fin de main()),
    //    tous les Livres seront automatiquement libérés (drop)
}
```

Ressources et bonnes pratiques

🔗 Ressources en ligne

1. Documentation officielle de Vec
2. Rust by Example - Vectors
3. The Rust Programming Language - Vectors

💡 Bonnes pratiques

1. **Pré-alouer la capacité** quand on connait la taille approximative (`with_capacity`)
2. **Utiliser `get`** plutôt que l'indexation directe pour éviter les paniques
3. **Préférer l'itération** aux accès par index quand possible
4. **Utiliser les énumérations** pour des collections hétérogènes
5. **Implémenter `Display`** pour un affichage personnalisé

⚠️ Pièges courants

1. **Oublier `mut`** quand on veut modifier un vecteur
2. **Accéder à un index hors limites** avec la syntaxe `[]`
3. **Tenir des références** pendant une modification du vecteur
4. **Négliger la capacité** pour des vecteurs qui vont grandir

---

📝 Exercices pratiques à partir du code d'exemple

1. Trier les livres par année de publication.
2. Implémenter une fonction qui retourne les publications à partie d'une année de référence.
3. Écrire une fonction qui compte le nombre de références par auteur dans une bibliographie complète.